

Square_Eyes-cryptohack

cryptography

Analisis

deskripsi soal:

It was taking forever to get a 2048 bit prime, so I just generated one and used it twice.

If you're stuck, look again at the formula for Euler's totient.

Challenge files:

- [output.txt](#)

isi output.txt:

n =

```
535860808044009550029177135708168016201451343147313565371014459027743491739
422885443084705720731409713775527993719682583669164873806842043288439828071
789970694759080842162253955259590552283047728782812946845160334801782088068
154453021936721710269050985805054692096738777321796153384024897615594493453
068138341203673749514094546000253631902991617197847584519694152122765406982
133526594928685232381934742152195861380221224370858128736975959176861651044
370378539093990198336298572944512738570839396588590096813217791191895941380
464803377602779240663133834952329316862399581950590588006371221334128215409
197603236942597674756728212232134056562716399155080108881105952768189193728
827484667349378091100068224404684701674782399200373192433062767622841264055
426035349769018117299620554803902490432339600566432246795818167460916180647
394169157647245603555692735630862148715428791242764799469896924753470539857
080767170052783918273180304835318388177089674231640910337743789750979216202
573226794240332797892868276309400253925932223895530714169648116569013581643
192341931800785254715083294526325980247219218364118877864892068185905587410
977152737936310734712276956663192182487672474651103240004173381041237906849
437490609652395748868434296753449
```

e = 65537

ct =

```
222502885974182429500948389840563415291534726891354573907329512556439632810
921927905220486727807436668035929302442754225952786602492250448020341217733
646472982286222338860566076161977786095675944552232391481278782019346283900
959677167026636830252067048759720251671811058647569724495547940966885025629
807079171218371644528053562232396674283745310132242492367274184667845174514
466834132589971388067076980563188513333661165819462428837210575342101036356
974189393390097403614434491507672459254969638032776897417674577487775755539
964915035731988499983726435005007850876000232292458554577437739427313453671
492956668188219600633325930981748162455965093222648173134777571527681591366
```

164711307355510889316052064146089646772869610726671696699221157985834325663
661400034831442431209123478778078255846830522226390964119818784903330200488
705212765569163495571851459355520398928214206285080883954881888668509262455
490889283862560453598662919522224935145694435885396500780651530829377030371
611921181207362217397805303962112100190783763061909945889717878397740711340
114311597934724670601992737526668932871436226135393872881664511222789565256
059138002651403875484920711316522536260604255269532161594824301047729082877
262812899724246757871448545439896

di deskrpsi soal kita di kasih tau bahwa nilai dari n adalah angka prima 2048 bit yang digunakan berulang.

It was taking forever to get a 2048 bit prime, so I just generated one and used it twice.

yang bisa di artikan bahwa

$n = 2048\text{bit_prima}^{2}$**

yang artinya **$p=q$** .

Solusi

pertama kita coba masukan int dari n ke <https://factordb.com> untuk mencoba mencari faktor dari n

Search	Sequences	Report results	Factor tables	Status	Downloads	API
53586080804400955002917713570816801620145134314731356537101445902774349173942288544308470572073140971: Factorize!						
Result:						
status (?)	digits	number				
FF	1233 (show)	$(2314866752...07_{<617>})^2 = (2314866752...07_{<617>})^2$				

di bawah adalah hasil dari server dan interger dari n kita statusnya **FF** yang artinya **Full Factorized**. karena kita sudah tau nilai p maka kita tinggal mencari ϕ_n untuk mendapatkan d, tapi karena **$n = p^{**2}$** maka untuk mencari ϕ_n kita rumusnya berubah menjadi **$\phi_n = p * (p-1)$**

solver:

```
from sympy import factorint
from Crypto.Util.number import long_to_bytes , inverse
from gmpy2 import iroot
n =
53586080804400955002917713570816801620145134314731356537101445902774349
17394228854430847057207314097137755279937196825836691648738068420432884
39828071789970694759080842162253955259590552283047728782812946845160334
80178208806815445302193672171026905098580505469209673877732179615338402
48976155944934530681383412036737495140945460002536319029916171978475845
```

```
19694152122765406982133526594928685232381934742152195861380221224370858
12873697595917686165104437037853909399019833629857294451273857083939658
85900968132177911918959413804648033776027792406631338349523293168623995
81950590588006371221334128215409197603236942597674756728212232134056562
71639915508010888110595276818919372882748466734937809110006822440468470
16747823992003731924330627676228412640554260353497690181172996205548039
02490432339600566432246795818167460916180647394169157647245603555692735
63086214871542879124276479946989692475347053985708076717005278391827318
03048353183881770896742316409103377437897509792162025732267942403327978
92868276309400253925932223895530714169648116569013581643192341931800785
25471508329452632598024721921836411887786489206818590558741097715273793
63107347122769566631921824876724746511032400041733810412379068494374906
09652395748868434296753449
```

```
e = 65537
```

```
ct =
```

```
22250288597418242950094838984056341529153472689135457390732951255643963
28109219279052204867278074366680359293024427542259527866024922504480203
41217733646472982286222338860566076161977786095675944552232391481278782
01934628390095967716702663683025206704875972025167181105864756972449554
79409668850256298070791712183716445280535622323966742837453101322424923
67274184667845174514466834132589971388067076980563188513333661165819462
42883721057534210103635697418939339009740361443449150767245925496963803
27768974176745774877757555399649150357319884999837264350050078508760002
32292458554577437739427313453671492956668188219600633325930981748162455
96509322264817313477757152768159136616471130735551088931605206414608964
67728696107266716966992211579858343256636614000348314424312091234787780
78255846830522226390964119818784903330200488705212765569163495571851459
35552039892821420628508088395488188866850926245549088928386256045359866
29195222249351456944358853965007806515308293770303716119211812073622173
97805303962112100190783763061909945889717878397740711340114311597934724
67060199273752666893287143622613539387288166451122278956525605913800265
14038754849207113165225362606042552695321615948243010477290828772628128
99724246757871448545439896
```

```
a, benar = iroot(n, 2)
```

```
if benar:
```

```
    print(f"nilai 2048 = {a}")
```

```
p =
```

```
23148667521998097720857168827790771337662483716348435477360567409355026
16916593444694980966459552377085389720310375910698398511326404905741690
81911667200085032759516257389756660190291723776531706024403735795932925
```

```
76530667773951407647222757756437867216095193174201323278896027294517792
60788186185526460052577246074525944030115693094325524091568571855233419
22302647803557991790378160263307054224840000865423620840069581585503463
95941862383925942033730030004606360308379776255436206440529441711859246
81158665274602841849602014544151303753547538096256210892069992902290067
7901988508936509354385660735694568216631382653107 #nilai yg kita dapat
dari factordb atau kita bisa cari dengan iroot(n,2) seperti yang ada di
atas
```

```
phi_n = p*(p-1)
```

```
d = inverse(e,phi_n)
```

```
m = pow(ct,d,n)
```

```
print(long_to_bytes(m))
```